

SmartTaps: Voice-Controlled Interaction for Smartphones

1st Michael Calderin

Department of Engineering Education

University of Florida

Gainesville, FL, USA

michaelcalderin@ufl.edu

Abstract—SmartTaps is a software program that enables users to control their devices entirely through voice. It converts spoken commands into taps on the screen using a speech-to-text model, multimodal vision-language understanding, and automated UI execution. This technical blueprint details datasets, preprocessing, model architecture, interactive interface, innovation, implementation timeline, and responsible AI considerations for the semester-long project.

I. PROBLEM CONTEXT AND PROJECT SUMMARY

In many ways, smartphones have become a daily essential, yet interacting with them is not always convenient and at times simply impossible. Consider having your hands occupied while cooking or washing dishes; tapping a screen with wet hands rarely produces the intended result. Touch-based smartphone controls can also be challenging for users with hand tremors or other physical disabilities. Current voice assistants often prove frustrating, requiring rigid voice commands and offering limited awareness of on-screen elements. SmartTaps addresses these issues by enabling users to control their phones entirely through voice, like other voice assistants, but with a visual understanding of the user interface that is not tied to specific app APIs or predefined commands. Spoken instructions such as “Open Spotify” are translated into screen coordinates for tapping, enabling seamless, hands-free interaction. The current proof of concept focuses on tap actions, but the architecture supports future extensions to gestures such as swiping, scrolling, or drag-and-drop through additional training.

II. DATASET

A. Audio-to-Text Evaluation Dataset

- **Source:** LJSpeech-1.1 [1], [2]
- **Type:** Audio and text (transcriptions)
- **Size:** 13.1k audio samples (1–10 second clips) with corresponding transcriptions, totaling approximately 3.56 GB
- **Format:** .wav for audio clips and .csv for metadata (transcriptions)
- **Access:** Manual download

This dataset will serve as a benchmark to assess the transcription power of the pre-trained audio-to-text model. The model will likely not require additional training; the primary focus of this project is on training the vision-language model,

as it represents the most significant innovation. However, evaluating the audio-to-text model remains important, since incorrect transcriptions could negatively impact the results.

To be compatible with an audio-to-text model like Whisper, there will need to be some preprocessing to convert to the required sampling rate. However, features that are typically problematic, such as noise, are actually beneficial for robustness. Still, there are concerns regarding the nature of the audio clips: they all seem to come from the same woman, and the clips are relatively short. This may lead to misleading evaluation results, as the test does not cover male voices or other accents. Additionally, the audio is drawn from novels, which differs from the natural commands a human would use on their phone. If time permits, a more diverse dataset will be used; however, this dataset was chosen for its cleanliness, simplicity, and low memory requirements, allowing more time to focus on the vision-language model.

B. Screen Annotation Dataset

- **Source:** Google Research Screen Annotation Dataset (derived from the RICO dataset) [3], [4]
- **Type:** Images with UI element annotations (bounding box, labels)
- **Size:** 22,417 annotated android phone screens
- **Format:** .jpg for screenshots and .csv for metadata (annotations)
- **Access:** Manual download

There are a few preprocessing challenges with this dataset. CLIP pre-trained requires 224x224 images so the strategy will be to pad images into squares before feeding them into the preprocessor that will handle resizing and normalization; this is in an effort to preserve the screen contents and prevent cropping UI elements. Also, the dataset is a solid foundation, but the vision-language model requires text commands and device screenshots as input, paired with coordinates of where to tap as output. The dataset lacks text commands so these will likely need to be synthetically generated with an LLM.

Ethical concerns related to bias may arise after training. For example, image-based UI elements in the training set might lack diversity, leading to weaker performance when asked to tap on images of minorities.

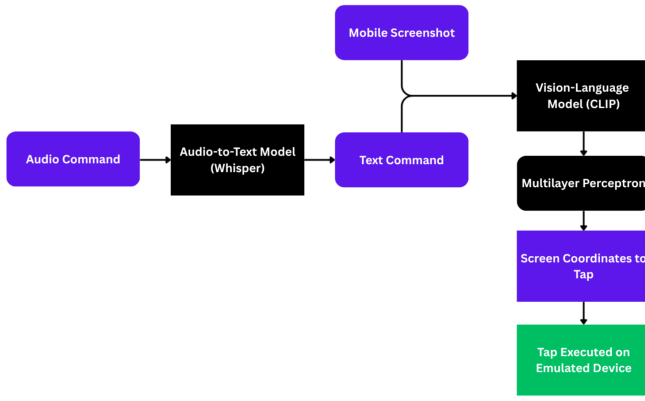


Fig. 1. Architecture and data flow of SmartTaps.

III. PLANNED ARCHITECTURE

Fig. 1 shows the planned architecture. Suppose this software were on a mobile device, running in the background. The user would speak into the microphone to give their voice command and the software would simultaneously and automatically take a screenshot. The audio would be converted into text by Whisper from Open AI. The text command and screenshot could then be fed to CLIP with a multilayer perceptron head to predict the screen coordinates of where to tap. The software would then automatically execute the tap at that location. For this project, the mobile device will be emulated and not physical.

The following will be required for various aspects of the system:

- PyTorch (Python library for model training)
- Hugging Face models (Whisper, CLIP)
- Appium (Python library for tap execution)
- Android Studio Emulator (allows computer to emulate mobile device)

As a whole, there are two main components to the system: a Python script and Android Studio Emulator. The Python script acts as a background process that will handle voice input, predictions, and tapping; it is the core logic and manager. The Android Studio Emulator will emulate a mobile device so that when the script signals a tap, the effects of the tap will be shown in the studio (e.g. an app is tapped and launches). In practice, the software would be implemented as a mobile app or integrated directly into the device's UI, capable of running in the background. However, this is a prototype designed to run entirely on a computer; the emulator is used so that a physical mobile device is not needed.

A potential challenge is understanding when the user starts and stops speaking. A naive approach will be employed where the amplitude of sound waves coming from the microphone will be monitored live. If the amplitude dampens for a certain period of time, the voice command will be considered complete and it will be sent through the pipeline.

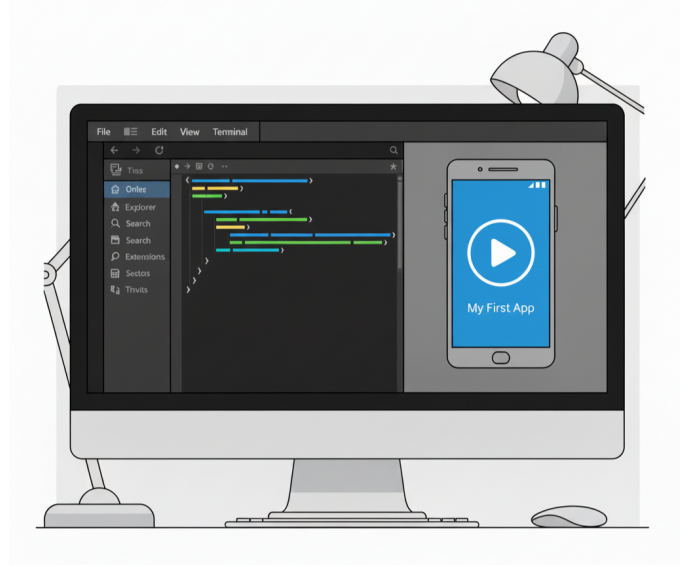


Fig. 2. Mockup of UI with Python script (running on VS Code) and emulated android device.

IV. USER INTERFACE PLAN

Fig. 2 shows a mockup of how a demo of the final system would look like. The user would run the Python script and then leave it running in the background. They could simply speak into the microphone and those voice commands would be used as input. The user would then notice a tap executed on the emulated mobile device. Although the interface would not work for production, the foundation is there for a product that runs in the background and requires minimal effort to interact with. For demonstration purposes, the emulator studio will be great to visualize how a phone would react to the user's voice command.

V. INNOVATION AND ANTICIPATED CHALLENGES

A. Innovation

SmartTaps is unique because it understands the visual content of the screen directly, rather than relying on pre-registered UI elements, app APIs, or rigid command structures. This allows users to give natural language instructions, such as "Click the blue button," without needing to memorize fixed commands or rely on app-specific integrations. By combining voice recognition with visual understanding, SmartTaps can flexibly interact with virtually any application interface. The key technical innovations include:

- **Screen-Aware Multimodal Interaction:** The system fuses spoken commands with on-screen visual context to predict tap coordinates, enabling natural, context-aware interactions.
- **End-to-End Automation:** From voice input to automatic tap execution on an emulated device, SmartTaps demonstrates a full hands-free control pipeline.
- **Synthetic Dataset Generation:** Since labeled voice-to-coordinate data does not exist, a dataset is generated by

pairing synthetic natural language commands with annotated screenshots, leveraging LLMs to produce realistic interactions.

B. Key Technical Risks and Mitigation

- 1) **Multimodal Pipeline Failure:** SmartTaps relies on multiple components, including audio-to-text transcription, vision-language understanding, and tap execution, working together seamlessly. If any component fails, the whole system fails. To mitigate this, each module will be tested independently before integration, and logging will be implemented to quickly identify errors.
- 2) **Weak Generalization:** The system must handle a wide variety of phone screens, but CLIP was not trained specifically on UI layouts and may fail on unfamiliar designs. To mitigate this, the training process will include a diverse set of layouts, and data augmentation techniques, such as color adjustments, will be applied to help the model learn abstract, layout-independent patterns.
- 3) **User Command Variability:** Variations in accents, speech patterns, or phrasing could cause transcription or understanding errors. To mitigate this, a robust pre-trained model such as Whisper will be used for transcription, and synthetic variations of commands will be generated to improve robustness to different speech styles.

VI. IMPLEMENTATION TIMELINE

Table I presents a week-by-week plan for completing the project from October 20 to December 11, outlining the primary focus and expected outcomes for each stage.

TABLE I
PROJECT TASKS AND TIMELINE

Week	Focus	Expected Outcome
Oct 20–26	Transform annotated phone screen dataset into training data with synthetic text commands and screenshots as inputs, tap coordinates as labels	Prepared dataset ready for vision-language model training
Oct 27–Nov 2	Baseline model setup and initial training	Evaluated speech-to-text model and preliminary multimodal model results
Nov 3–16	Model tuning and performance improvement	Enhanced performance with refined architecture and hyperparameters
Nov 17–30	Integration of microphone and emulator	Functional system mapping voice input to simulated taps on device screens
Dec 1–11	Final testing, demo, and reporting	Completed demonstration, final report, and presentation materials

VII. RESPONSIBLE AI REFLECTION

SmartTaps raises several responsible AI concerns. Fairness is a concern because the system may perform differently across accents, genders, and speech styles, potentially reflecting biases in the training data. Transparency is a concern since users may not easily understand how voice commands are translated into screen taps. Accessibility is a concern because users with motor impairments or other physical limitations may experience varying levels of benefit. Environmental impact is

also a concern due to the energy consumption involved in training and running the models.

REFERENCES

- [1] “The lj speech dataset,” <https://www.kaggle.com/datasets/mathurinache/the-lj-speech-dataset>, accessed: 2025-10-16.
- [2] “Ljspeech sr16k dataset,” <https://www.kaggle.com/datasets/awsaf49/ljspeech-sr16k-dataset>, accessed: 2025-10-16.
- [3] “Google research screen annotation dataset,” https://github.com/google-research-datasets/screen_annotation, accessed: 2025-10-16.
- [4] “Rico dataset,” <https://www.interactionmining.org/archive/rico#quick-downloads>, accessed: 2025-10-16.